CAD for VLSI Design
Programming Assignment 2

Partitioning Project Report

Advisor: Yu-Guang Chen

Student Info: 電機 4A

Student ID: 110501521

Student Name: 徐松廷

1. How to Compile and Execute My Program

a. My Makefile

依照助教要求，本次 PA2 會需要按照以下方式，利用 makefile 支援 make all、make run、make clean，編譯.cpp file 並執行.exe file，並接收使用者的 input file name 以及 output file name 執行.exe，最終產生 partition 的結果，再利用 make clean 清除執行檔。

```
# PA2 cpp compiler

.PHONY: all run clean

all: exe

exe: 110501521_PA2.o
	@g++ -std=c++11 110501521_PA2.o -o exe

run:
	@./exe $(input) $(output)

clean:
	@rm -f *.o exe

110501521_PA2.o: 110501521_PA2.cpp
	@g++ -std=c++11 -c 110501521_PA2.cpp
```

b. Compiling and Executing Steps:

依照助教要求，本次 PA2 會需要按照以下方式，利用 makefile 去編譯.cpp file 並執行.exe file，產生.out 後利用助教提供的 checker 確認我們的 Partition 結果是否正確，詳細步驟如下:

I. Step 1: make all

II. Step 2: make run input=case output=case.out

III. Step 3: ./PA2_CHECKER case2 case2.out

IV. Step 4: make clean

2. The Completion of The Assignment

a. Partition Result 符合 balance constraint:本次 PA 的 Constraint 如下

Constraints

- |size(A) – size(B)| <= n/5, where n is the number of cells in the circuit and n always >= 10

我的實作方式中，每次選擇需移動的 cell 時，都會用以下 function 確認要從 A 移動 cell 還是從 B 移動 cell 還是從 A 與從 B 移動 cell 皆可:

```
Function check_move_from_A_or_B(Output: moveA, Output: moveB, Input: partition):
    countA ← 0
    countB ← 0
    For each cell in partition:
        If cell in "A": countA ← countA + 1
        Else:           countB ← countB + 1

    total ← countA + countB
    limit ← total / 5  // 平衡允許的最大差距（20%）
    moveB ← abs((countA + 1) - (countB - 1)) ≤ limit
    moveA ← abs((countB + 1) - (countA - 1)) ≤ limit
End Function
```

b. Minimize Cut Size with FM Algorithm:本次 PA 要求基於 bucket list，以 FM Algorithm 實作 Partition 並最小化 Cut Size，我實作的 Function 演算法如下

```
Function Fiduccia_Mattheyses(cell_set, net_set, outputFilename):

    cur_best_partition ← initialize_partition(cell_set)
    history_best_partition ← cur_best_partition
    partition_result ← cur_best_partition

    cur_best_cutsize ← calculate_cutsize(net_set, cur_best_partition)
    history_best_cutsize ← cur_best_cutsize
    iteration_count ← 0

    Loop forever:
        // 根據當前 partition 結果建立 gain bucket list，所有 cell 尚未被鎖定
        bucket_list ← initialize_bucket_list(cell_set, net_set, partition_result)

        cur_cutsize ← calculate_cutsize(net_set, partition_result)

        // 更新當前最佳解
        cur_best_partition ← partition_result
        cur_best_cutsize ← cur_cutsize

        // Inner loop: 不斷選取最大 gain 的 cell 並移動
        Loop forever:
            gain ← 0
            cell_to_move ← find_max_gain_cell_under_balance_constraint(bucket_list, partition_result, gain)
```

```
            If cell_to_move is invalid OR gain < 0:
                Break inner loop

            // 移動 cell 並更新 gain 結構與 partition 結果
            cur_cutsize ← cur_cutsize - gain
            move_cell_and_update(cell_to_move, cell_set, net_set, bucket_list, partition_result)

            // 若此移動帶來更好 cutsize，更新當前最佳結果
            If cur_cutsize < cur_best_cutsize:
                cur_best_cutsize ← cur_cutsize
                cur_best_partition ← partition_result

        // Outer loop: 檢查這輪是否有更好結果
        If cur_best_cutsize < history_best_cutsize:
            history_best_cutsize ← cur_best_cutsize
            history_best_partition ← cur_best_partition
        Else:
            Break outer loop

        partition_result ← history_best_partition
        Print "cur_best_cutsize:", cur_best_cutsize, "history_best_cutsize:", history_best_cutsize
        iteration_count ← iteration_count + 1

    // 寫出最終結果
    write_file(outputFilename, history_best_cutsize, history_best_partition)
```

c. Write File with Correct Format: 本次 PA 要求再輸出檔需輸出優化後的 cut size 以及 Partition 後 cell 的分群結果，我們實作的結果符合要求，結果如下:

1. 正確計算 cutsize

2. 正確輸出 Partition 結果並符合 Balance 要求。

```
cut_size 3
A
c11
c10
c7
c1
B
c4
c8
c9
c6
c5
c2
```

3. Method of My Design - Data Structure

為了有效率地實作並執行 FM Algorithm，至少需要 Net Array、Cell Array、以及最關鍵的 Bucket List 加速 Gain 的更新，詳細資料結構如下:

a. Net Array

負責記錄所有 Net 連接的 Cell，計算 Cutsize、特定 Cell 的 Gain 以及更新 Cell 的 Gain 的時候會反覆使用到，教授上課是以 Array 的形式實現，**但因為 Net ID 可能不連續且需要在常數時間內查找，所以我以 unordered_map 實作，key 是 Cell ID，value 是所有聯的 cell id 形成的 unordered_set。**

Net array	
Net 1	C1, C2, C3, C4
Net 2	C1, C5
Net 3	C2, C5
Net 4	C3, C6
Net 5	C4, C6
Net 6	C4, C6

```
struct net_node{    //for  net_set
    string net_name;
    unordered_set<int> cell_id_set;
};
```

```
unordered_map<int, struct net_node> net_set;
```

b. Cell Array

負責記錄所有 Cell 連接的 Net，計算 Cutsize、特定 Cell 的 Gain 以及更新 Cell 的 Gain 的時候會反覆使用到，教授上課是以 Array 的形式實現，**但因為 Cell ID 可能不連續且需要在常數時間內查找，所以我以 unordered_map 實作，key 是 Net ID，value 是所有聯的 Net id 形成的 unordered_set。**

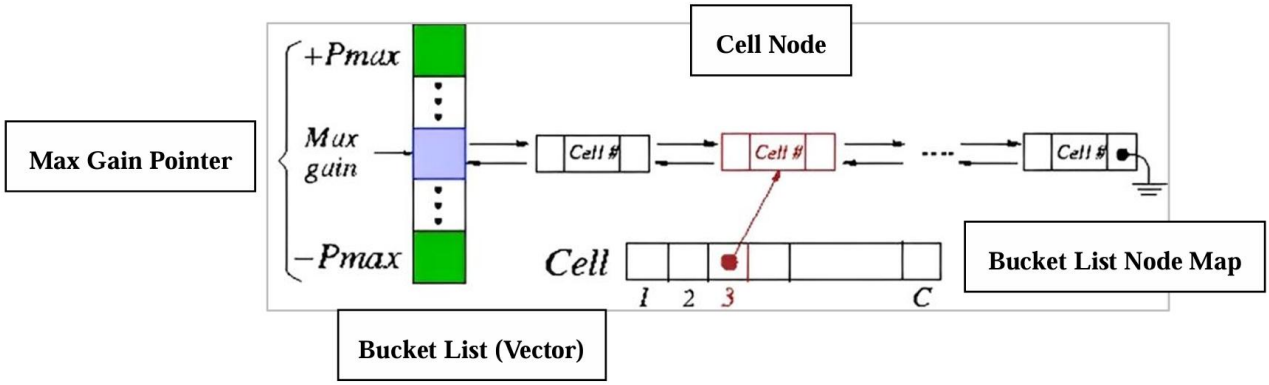
Cell array	
C1	Nets 1, 2
C2	Nets 1, 3
C3	Nets 1, 4
C4	Nets 1, 5, 6
C5	Nets 2, 3
C6	Nets 4, 5, 6

```
struct cell_node{    //for cell_set
    string cell_name;
    unordered_set<int> net_id_set;
};
```

```
unordered_map<int, struct cell_node> cell_set
```

c. **Bucket List**

FM 演算法關鍵的資料結構就是 Bucket List，記錄所有 cell node 的 gain 並安插在 Bucket List 的正確位置，並以 Max Gain Pointer 以及 Cell node Map 加速 Cell Node Map 的查找，**但老實說要實作 FM 演算法其實未必需要用 Bucket List，資料結構的部分只要確保 Cell Node 的 Gain 可以在常數時間內查找完畢，且依照 Gain 大小排序 cell node 即可(為了後續的 Max Gain Pointer 可以在常數時間內更新)，用 vector 或是 Max Heap 也行，重點是要確保移動 base cell 後，被影響到的 Cell Node 的 Gain 可以在 linear time 被正確更新，且初始化 cell 的 cut size 要夠快，否則會像我一開始的寫法一樣，在 testcase 2 跑到天荒地老、邊寫邊哭。**



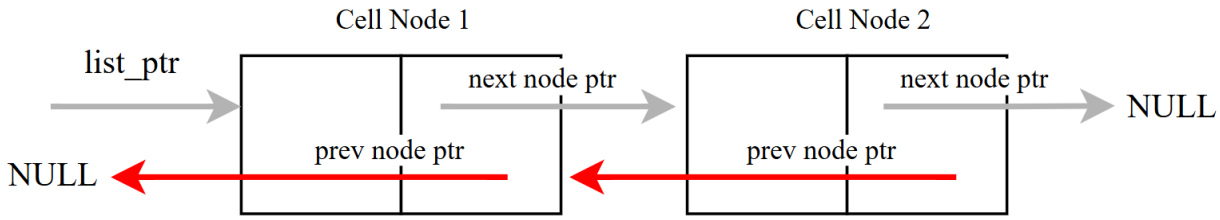
I. **Cell Node:** 在 Bucket List 中要把每個 cell 依照 gain 安插在正確位置，以下就是 Cell Node 的資料結構，包含 cell id、是否被鎖上、gain value 等。

```
struct cell_node_of_bucket_list{
    int gain;   //雖然從bucket list可以直接拿，但還是紀錄一下
    int cell_id;
    bool locked_bool;
    struct cell_node_of_bucket_list *next_cell_node_ptr;
    struct cell_node_of_bucket_list *prev_cell_node_ptr;
};
```

II. **Bucket List Node :** 在 Bucket List vector 每個 element 的資料結構，紀錄 Gain 以及指向第一個 node 的 pointer，資料結構如下

```
struct bucket_list_node{
    int gain;   //中間等於0
    struct cell_node_of_bucket_list *list_ptr;
};
```

因為在實作時稍有混淆，所以這邊特別做了一張圖記錄一下，**經過這次經驗稍微更熟悉雙向 linked list，重點是 pointer 必定會指到一個對應 data type 的變數，否則必為空指針:**

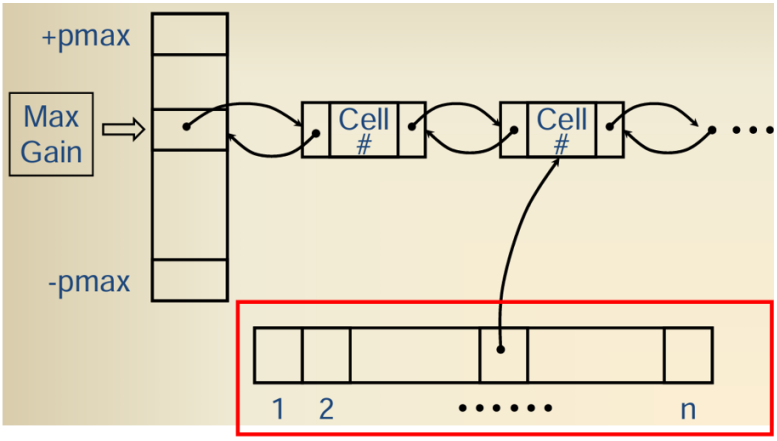


III. Bucket List (Vector)+Max Gain Pointer: Bucket List 的資料結構，包含 Bucket List (Vector)本身以及 Max Gain Pointer，指向 Gain 最大的 cell node。

```
struct bucket_list{
    struct cell_node_of_bucket_list* max_gain_node_ptr;
    vector <struct bucket_list_node> bucket_list;
};
```

IV. Bucket List Node Class: 最上層用以紀錄 Part A 以及 Part B 的 Bucket List 以及對應的 Cell Node Map，Cell Node Map 非常關鍵，可以以 Cell ID 在常數時間內拿到 Bucket List 中對應的 Cell Node，算是最關鍵的資料結構之一，也是以 unordered_map 實作。

```
struct bucket_list_class{
    struct bucket_list  bucket_list_part_A;
    unordered_map<int,struct cell_node_of_bucket_list*> cell_id_to_node_map_part_A;

    struct bucket_list  bucket_list_part_B;
    unordered_map<int,struct cell_node_of_bucket_list*> cell_id_to_node_map_part_B;
};
```



4. Method of My Design - Computation Steps of My Program

a. Step 1: Read Input File

根據輸入檔名，利用 getline( )讀檔，並初始化 Cell Set 與 Net Set 以利後續計算

```
void read_file(string input_file_name,unordered_map<int, struct cell_node>&cell_set,unordered_map<int, struct net_node>&net_set){
    vector<string> net_lines;
    ifstream file(input_file_name);

    if(!file){
        cerr<<"can't open file: "<<input_file_name<<endl;
        return ;
    }

    string line;
    while(getline(file, line)){ //使用 getline()以行為單位讀取
        net_lines.push_back(line);
    }
    file.close();
    construct_net_lines_to_cell_set_and_net_set (net_lines,cell_set,net_set);
}
```

b. Step 2: 利用 Fiduccia - Mattheyses Algorithm 優化 cut size

此步驟是 PA2 的核心步驟， 包含基於 Net Set、Cell Set、Bucket List 建構 bucket list，外部迴圈反覆更新 history best cut size，內部迴圈則是按照 gain 的大小移動所有 Cell 並上鎖，紀錄內部迴圈運行的最佳 partition，並更新 history best cut size，最後直到跑完一輪內部迴圈無法得到更好的 cut size 就跳出外部迴圈:

```
void Fiduccia_Mattheyses(unordered_map<int,struct cell_node>cell_set,
    unordered_map<int,struct net_node>net_set, unordered_map<int,string> &partition_result, string outputFilename){

    unordered_map<int,string> cur_best_partition=partition_result;  //只存best
    unordered_map<int,string> history_best_partition=partition_result;

    int cur_best_cutsize=calculate_cutsize(net_set, cur_best_partition);
    int history_best_cutsize=cur_best_cutsize;

    while(1){
        //根據當前的best_partition初始化gain_map (所有的cell都還沒被鎖上)
        struct bucket_list_class my_bucket_list_class=initialize_bucket_list_object(cell_set,net_set,partition_result);

        int cur_cutsize=calculate_cutsize(net_set, partition_result);

        cur_best_partition=partition_result;
        cur_best_cutsize=cur_cutsize;
        while(1){
            int gain_of_cell_id_to_move=0;
            int cell_id_to_move=find_max_gain_cell_under_balance_constraint(my_bucket_list_class,partition_result,gain_of_cell_id_to_move);
            cout<<"FM: selected cell_id_to_move: "<<cell_id_to_move<<endl;
            if(cell_id_to_move==-1||gain_of_cell_id_to_move<0)break;
            else{
                cur_cutsize-=gain_of_cell_id_to_move;
                move_cell_and_update_bucket_list_and_partition(cell_id_to_move,cell_set,net_set,my_bucket_list_class, partition_result);
                //如果當前的cutsize比cur_best_cutsize好(更新cur_best_cutsize以及partition)
                if(cur_cutsize<cur_best_cutsize){
                    cur_best_cutsize=cur_cutsize;
                    cur_best_partition=partition_result;
                }
            }
        }
        if(cur_best_cutsize<history_best_cutsize){ //不確定這樣的做法是不是最好
            history_best_cutsize=cur_best_cutsize;
            history_best_partition=cur_best_partition;
        }
        else break;
        partition_result=history_best_partition;
        cout<<"cur_best_cutsize: "<<cur_best_cutsize<<" history_best_cutsize: "<<history_best_cutsize<<endl;
    }
    write_file(outputFilename, calculate_cutsize(net_set, history_best_partition),  history_best_partition);
    cout<<"final result: "<<calculate_cutsize(net_set, history_best_partition)<<endl;
```

詳細步驟與相關函式如下:

I. Sub-Step 1: initialize_bucket_list_object( )

根據 Cell 中最大持有的 Net 數初始化 bucket list vector (size=2*max_gain+1)後，利

用動態記憶體配置建構 Cell Node，並計算每個 cell 的 gain，以上步驟完成後再將建構好的 Cell Node 插入到 bucket list 中的正確位置:

```
struct bucket_list_class initialize_bucket_list_object( unordered_map<int,struct cell_node>cell_set,
    unordered_map<int,struct net_node>net_set, unordered_map<int,string> partition_result){
    struct bucket_list_class my_bucket_list_class;

    int max_gain=0;             //找max_gain才能知道bucket list要開多大
    find_max_num_of_net(cell_set, max_gain,partition_result);

    resize_bucket_list(max_gain,my_bucket_list_class.bucket_list_part_A.bucket_list);  //用max_gain初始化bucket list的list本身
    resize_bucket_list(max_gain,my_bucket_list_class.bucket_list_part_B.bucket_list);

    for(auto it:cell_set){      //初始化每個cell的cell node
        int cell_id=it.first;
        int gain=calculate_gain_of_cell(cell_id,cell_set,net_set,partition_result);
        struct cell_node_of_bucket_list* my_cell_node_of_bucket_list_ptr=new cell_node_of_bucket_list;

        my_cell_node_of_bucket_list_ptr->cell_id=cell_id;
        my_cell_node_of_bucket_list_ptr->gain=gain;
        my_cell_node_of_bucket_list_ptr->locked_bool=false;
        my_cell_node_of_bucket_list_ptr->next_cell_node_ptr=NULL;
        my_cell_node_of_bucket_list_ptr->prev_cell_node_ptr=NULL;

        if(partition_result[cell_id]=="A"){     //把node插入bucket list要額外寫個function
            insert_cell_node_to_bucket_list(gain+max_gain,my_cell_node_of_bucket_list_ptr,my_bucket_list_class.bucket_list_part_A.bucket_list);
            my_bucket_list_class.cell_id_to_node_map_part_A[cell_id]=my_cell_node_of_bucket_list_ptr;
        }
        else{
            insert_cell_node_to_bucket_list(gain+max_gain,my_cell_node_of_bucket_list_ptr,my_bucket_list_class.bucket_list_part_B.bucket_list);
            my_bucket_list_class.cell_id_to_node_map_part_B[cell_id]=my_cell_node_of_bucket_list_ptr;
        }
    }

    my_bucket_list_class.bucket_list_part_A.max_gain_node_ptr = find_max_gain_node_ptr(my_bucket_list_class.bucket_list_part_A.bucket_list);
    my_bucket_list_class.bucket_list_part_B.max_gain_node_ptr = find_max_gain_node_ptr(my_bucket_list_class.bucket_list_part_B.bucket_list);
    return my_bucket_list_class;
}
```

i. **Step 1-1 初始化 Cell Node:利用動態記憶體配置索取記憶體空間。**

ii. **Step 1-2 計算該 Cell 的 Gain**

這個步驟會遍歷該 Cell 持有的所有 Net，並計算所有 Net 的 From 和 To，如果假設 worst time complexity 約為 O(N*C)，N 為該 cell 的所有 Net 數，C 為一個 Net 持有 Cell 數的上限。

```
int calculate_gain_of_cell(int cell_id,unordered_map<int,struct cell_node>cell_set,
    unordered_map<int,struct net_node>net_set,unordered_map<int,string> &patition_result){
    int gain=0;
    string From_part=patition_result[cell_id];
    for(auto net_id: cell_set[cell_id].net_id_set){
        int From=0;int To=0;
        calculate_From_and_To(net_id,From_part,net_set,patition_result,From,To);
        if(From==1)gain++;
        if(To==0)gain--;
    }
    return gain;
}
```

```
void calculate_From_and_To(int net_id,string From_part,
    unordered_map<int,struct net_node>net_set,unordered_map<int,string> partition_result,int &From,int &To){
    From=0;
    To=0;
    for(auto related_cell_id:net_set[net_id].cell_id_set){
        if(partition_result[related_cell_id]==From_part)From++;
        else To++;
        if(From>1&&To>0){
            From=-1;To=-1;
            break;
        }
    }
}
```

我的做法參考自教授上課的講義:

Initialization of all cell gains requires $O(P)$ time:

$g(i) \leftarrow 0$;

$F \leftarrow$ the "from block" of Cell i;

$T \leftarrow$ the "to block" of Cell i;

for each net n on Cell i **do**

  if $F(n)=1$ **then** $g(i) \leftarrow g(i)+1$;

  if $T(n)=0$ **then** $g(i) \leftarrow g(i)-1$;

iii. **Step 1-3 insert_cell_node_to_bucket_list:** 直接將 Cell Node 插在 Bucket List 中 gain 對應 doubly linked list 的開頭，此步驟只需常數時間。

```
void insert_cell_node_to_bucket_list(int index,
    struct cell_node_of_bucket_list* my_cell_node_of_bucket_list, vector<struct bucket_list_node> &bucket_list){
    //直接插在頭就好
    struct cell_node_of_bucket_list * node_ptr=bucket_list[index].list_ptr;
    if(bucket_list[index].list_ptr==NULL){      //當前要插入的node為head，且原本沒有任何node，故prev_node_ptr=NULL
        my_cell_node_of_bucket_list->next_cell_node_ptr=NULL;
        bucket_list[index].list_ptr=my_cell_node_of_bucket_list;
        my_cell_node_of_bucket_list->prev_cell_node_ptr=NULL;
    }
    else{                                       //當前要插入的node為head，且原本有node，故prev_node_ptr=NULL
        my_cell_node_of_bucket_list->next_cell_node_ptr=bucket_list[index].list_ptr;
        bucket_list[index].list_ptr->prev_cell_node_ptr=my_cell_node_of_bucket_list;

        bucket_list[index].list_ptr= my_cell_node_of_bucket_list;
        my_cell_node_of_bucket_list->prev_cell_node_ptr=NULL;
    }
}
```

II. **Sub-Step 2: find_max_gain_cell_under_balance_constraint( )** 確認依照目前的 partition 狀況，必須要從 A 移動到 B 還是 B 移動到 A，或是兩種皆可，並利用 Part A 與 Part B 的 Max Gain Pointer 選出合適的最大 Gain Cell。

```
int find_max_gain_cell_under_balance_constraint(struct bucket_list_class my_bucket_list_class,unordered_map<int,string> partition_result,int &cell_gain){
    bool can_move_from_A_bool,can_move_from_B_bool;   //每次都要從part_A的max gain或是part_B的max gain指到的node當作要移動的cell
    check_move_from_A_or_move_from_B(can_move_from_A_bool,can_move_from_B_bool,partition_result);
    string choice="NONE";//NONE or A or B

    if(can_move_from_A_bool==false&&can_move_from_B_bool==false){
        choice="NONE"; //這邊理論上不會發生，沒有合法解
    }
    else if(can_move_from_A_bool==true&&can_move_from_B_bool==true){ //從兩邊移動都是合法的
        if(my_bucket_list_class.bucket_list_part_A.max_gain_node_ptr!=NULL&&my_bucket_list_class.bucket_list_part_B.max_gain_node_ptr!=NULL){
            //兩者皆可選(選gain大的)
            if(my_bucket_list_class.bucket_list_part_A.max_gain_node_ptr->gain>my_bucket_list_class.bucket_list_part_B.max_gain_node_ptr->gain)choice="A";
            else choice="B";
        }
        else if(my_bucket_list_class.bucket_list_part_A.max_gain_node_ptr)choice="A";
        else if(my_bucket_list_class.bucket_list_part_B.max_gain_node_ptr)choice="B";
        else choice="NONE";//沒有合法解
    }
    else if(can_move_from_A_bool==true&&can_move_from_B_bool==false&&my_bucket_list_class.bucket_list_part_A.max_gain_node_ptr!=NULL)choice="A";//代表要從A移動到B才會符合限制
    else if(can_move_from_B_bool==true&&can_move_from_A_bool==false&&my_bucket_list_class.bucket_list_part_B.max_gain_node_ptr!=NULL)choice="B";//代表要從B移動到A才會符合限制
    else choice="NONE";//沒有合法解

    if(choice=="A"){
        cell_gain=my_bucket_list_class.bucket_list_part_A.max_gain_node_ptr->gain;
        return my_bucket_list_class.bucket_list_part_A.max_gain_node_ptr->cell_id;
    }
    else if(choice=="B"){
        cell_gain=my_bucket_list_class.bucket_list_part_B.max_gain_node_ptr->gain;
        return my_bucket_list_class.bucket_list_part_B.max_gain_node_ptr->cell_id;
    }
    else{
        cell_gain=0;//不選擇
        return -1;
    }
}
```

III. **Sub-Step 3: move_cell_and_update_bucket_list_and_partition( )** 此 Function 在選定要移動的 Base Cell 後，移動 cell 並更新 bucket list 與 partition 的主函式，根據 cell 所在區塊（A 或 B），執行以下操作：

```
void move_cell_and_update_bucket_list_and_partition(int cell_id_to_move,unordered_map<int,struct cell_node>cell_set,
    unordered_map<int,struct net_node>net_set,struct bucket_list_class &my_bucket_list_class, unordered_map<int,string> &partition_result){

    if(partition_result[cell_id_to_move]=="A"){   //更新相關的cell的gain以及在位置(這邊的map不用動，因為part A的cell還是那些，part B的cell也還是那些)
        struct cell_node_of_bucket_list* cell_node_to_move=my_bucket_list_class.cell_id_to_node_map_part_A[cell_id_to_move];
        update_all_the_related_cell_gain(cell_id_to_move,cell_set,net_set,my_bucket_list_class, partition_result);

        partition_result[cell_id_to_move]="B"; //partition已更新 //把cell_id從A移動到B

        //重新計算cell_id_to_move的gain並移動該node，並上鎖(上鎖部分確定OK)，這邊的map要動，因為part A map的cell ptr會被刪除，part Bmap的cell ptr會增加
        int new_gain=calculate_gain_of_cell(cell_id_to_move,cell_set,net_set,partition_result);
        move_cell_and_update_gain(cell_id_to_move,new_gain, cell_set, net_set,true, my_bucket_list_class.cell_id_to_node_map_part_A, my_bucket_list_class.bucket_list_part_A,

        my_bucket_list_class.cell_id_to_node_map_part_B[cell_id_to_move]=my_bucket_list_class.cell_id_to_node_map_part_A[cell_id_to_move];
        my_bucket_list_class.cell_id_to_node_map_part_A.erase(cell_id_to_move);

        update_max_gain_ptr_A_and_B(my_bucket_list_class);//更新max_gain_ptr

    }
    else{    //更新相關的cell的gain以及在位置
        struct cell_node_of_bucket_list* cell_node_to_move=my_bucket_list_class.cell_id_to_node_map_part_B[cell_id_to_move];

        update_all_the_related_cell_gain(cell_id_to_move,cell_set,net_set,my_bucket_list_class, partition_result);

        partition_result[cell_id_to_move]="A"; //partition已更新 //把cell_id從B移動到A

        //重新計算cell_id_to_move的gain並移動該node，並上鎖(上鎖部分確定OK)
        int new_gain=calculate_gain_of_cell(cell_id_to_move,cell_set,net_set,partition_result);
        move_cell_and_update_gain(cell_id_to_move,new_gain, cell_set, net_set,true, my_bucket_list_class.cell_id_to_node_map_part_B, my_bucket_list_class.bucket_list_part_B,

        my_bucket_list_class.cell_id_to_node_map_part_A[cell_id_to_move]=my_bucket_list_class.cell_id_to_node_map_part_B[cell_id_to_move];
        my_bucket_list_class.cell_id_to_node_map_part_B.erase(cell_id_to_move);

        update_max_gain_ptr_A_and_B(my_bucket_list_class);//更新max_gain_ptr
    }
}
```

i. **Step 3-1 更新 related cell 的 gain 值**: 呼叫 update_all_the_related_cell_gain( )，根據被移動 cell 所在的 net，更新所有和 base cell 持有相同 net 的 cell 的 gain。

ii. **Step 3-2 修改 partition 結果**: 將 cell_id_to_move 從 A 移到 B，或從 B 移到 A，實作方式參考次講義，完整的 code 如下:

```
int initialize_with_SA(unordered_map<int,struct cell_node>cell_set,
    unordered_map<int,struct net_node>net_set, unordered_map<int,string> &partition_result, double initialTemp, double coolingRate, int iterations){
    int cur_bestCutSize = calculate_cutsize(net_set,partition_result);
    unordered_set<int> part_A_cell_set;
    unordered_set<int> part_B_cell_set;
    construct_part_A_B_set(partition_result,part_A_cell_set, part_B_cell_set);
    double temp = initialTemp;
    srand(time(0));
    for(int i=0;i<3;i++){
        for (int iter = 0; iter < 1; iter++){      //選擇一個節點進行翻轉，以當前Partition為基礎去調整
            string chosen_part="None";
            int cell_index= get_random_cell_id_from_certain_part(cell_set, part_A_cell_set, part_B_cell_set,chosen_part);

            int gain_of_the_selected_cell=calculate_gain_of_cell_for_SA(cell_index,cell_set,net_set,partition_result);
            cout<<"gain_of_the_selected_cell: "<<gain_of_the_selected_cell<<endl;//已確認正確

            int newCutSize=cur_bestCutSize-gain_of_the_selected_cell;            //計算新的 cut-size
            int delta=-gain_of_the_selected_cell;   // 計算接受機率

            if(delta<0){                            // 接受新解(更新)
                cur_bestCutSize = newCutSize;
                if(chosen_part=="A"){
                    part_A_cell_set.erase(cell_index);
                    part_B_cell_set.insert(cell_index);
                    partition_result[cell_index]="B";
                }
                else{
                    part_B_cell_set.erase(cell_index);
                    part_A_cell_set.insert(cell_index);
                    partition_result[cell_index]="A";
                }
            }
            temp *= coolingRate;  // 降溫
        }
    }
    cout<<"cutsize after SA: "<<cur_bestCutSize<<endl;
    return cur_bestCutSize;
}
```

iii. **Step 3-3 重新計算被移動 cell 的 gain 值**: 用 calculate_gain_of_cell() 計算該 cell 在新 partition 下的 gain。

iv. **Step 3-4 更新 bucket list**: 呼叫 move_cell_and_update_gain() 將該 cell 從來源 bucket list 中移除並插入目標 bucket list，並將其標記為 locked。

v. **Step 3-5 更新 cell_id 對應的 map**: 根據 cell 移動的方向更新

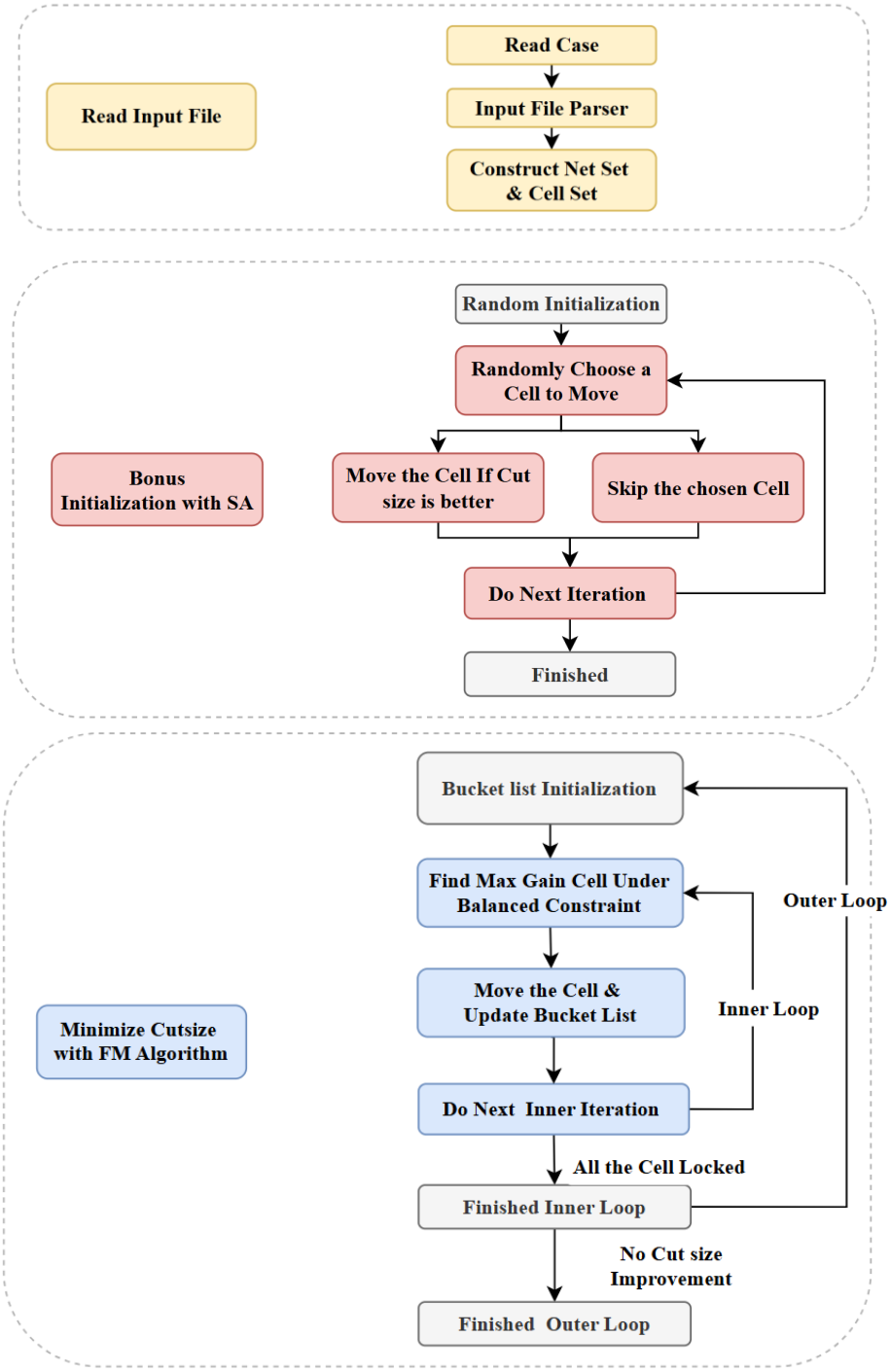
cell_id_to_node_map。

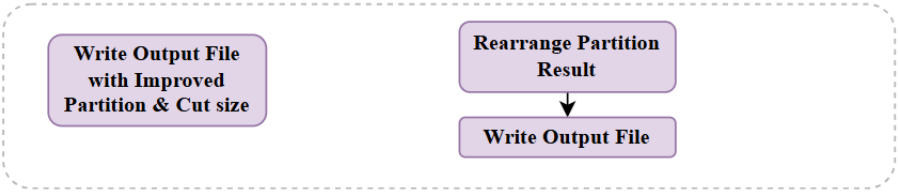
vi. **Step 3-6 更新 max gain pointer**: 最後呼叫 update_max_gain_ptr_A_and_B() 維護每個 partition 中最大 gain 的 cell 位置，方便之後選擇最佳移動。

IV. **Sub-Step 4 update cut size if current partition is better**: 如果在內部迴圈有得到比 current best cut size 更好的 cut size 就紀錄該 Partition 與 cut size。

V. **Sub-Step 5 go back to Sub-Step 2:** 回到 Sub-Step 2 繼續執行下一次內部迴圈。

5. Method of My Design - Flow Chart of My Program





首先從輸入檔案讀取電路資訊，並解析建立元件集合(Cell Set)與網路集合(Net Set)接著進行初始分割，流程中額外加入模擬退火(Simulated Annealing)方法作為加強版初始化，以隨機選擇元件並嘗試移動至另一分區，根據 cut size 變化決定是否接受，藉由控制溫度逐步收斂至更佳解。之後進入 FM (Fiduccia-Mattheyses)演算法階段，透過 bucket list 儲存不同增益的元件，反覆尋找最大增益且符合平衡限制的元件進行移動，並更新資料結構與 cut size，直到所有元件都被鎖定或無法改善 cut size 為止。最後將最佳的分割結果與 cut size 寫入輸出檔案中，完成整體切割最佳化流程。

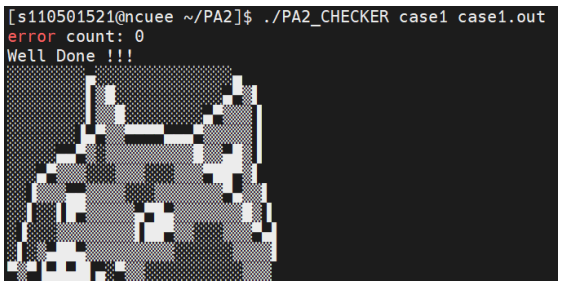
6. Test Cases Results

a. Public Case 1

I. Output File: cut_size=3

```
cut_size 3
A
c11
c10
c7
c1
B
c4
c8
c9
c6
c5
c2
```

II. Checker Result



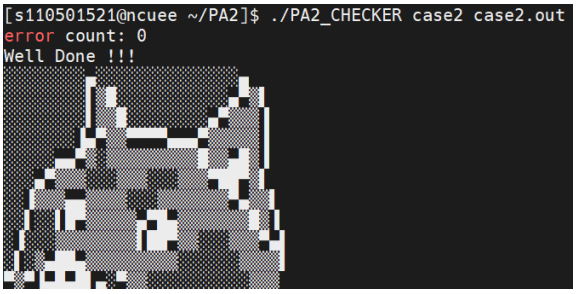
b. Public Case 2

I. Output File cut size = 887

```
cut_size 887
A
c11
c3
c161
c48
c87
c139
c50
c61
c39
c107
c114
c13
c127
c34
c136
c89
c185
c166
c54
c151
c111
c155
```

```
B
c106
c154
c51
c191
c47
c62
c60
c100
c32
c70
c163
c68
c174
c153
c156
c77
c46
c58
c33
c182
c196
c186
c145
```

II. Check Result



7. Bonus Implementation: Simulated Annealing

a. Simulated Annealing

依照 PA2 的要求，Bonus 的部分希望我們以優於隨機化的方式初始化 Partiton 以利後續 FM 演算法的更新，選用 SA 的原因是基於 SA 的特性，SA 每次接受新解的邏輯如下:

$$P_{\text{accept}} = \begin{cases} 1, & \text{if } \Delta E \leq 0 \\ e^{-\Delta E/T}, & \text{if } \Delta E > 0 \end{cases}$$

如果當前的解已優於原本的解，則無條件更新，如果當前的解差於原本的解，也會有一定的機率更新，而接受較差的解的機率如下，可以確保在溫度較仍較高時，接受新解的機率較高，較容易找到 global minimal，當溫度下降時就逐步降低接受較差的解的機率:

$$P_{\text{accept}} = \exp\left(-\frac{\Delta E}{T}\right)$$

Simulated Annealing 的在數學上可被證明在初始溫度無窮大且 iteration 次數無窮多的時候，可以得到 global optimal，因為 Simulated Annealing 的接受機率符合波茲曼分布，令機率總合為 1:

$$\sum_{x \in S} \pi_T(x) = 1 \qquad \pi_T(x) = \frac{e^{-E(x)/T}}{Z_T}, \quad \text{其中 } Z_T = \sum_{x \in S} e^{-E(x)/T}$$

當溫度趨近於 0 時，可得:

$$\pi_T(x^*) = \frac{e^{-E(x^*)/T}}{Z_T} \to 1$$

所有其他 $x \neq x^*$，有 $E(x) > E(x^*)$，故 $e^{-E(x)/T} \ll e^{-E(x^*)/T}$

所以總 partition function $Z_T \approx e^{-E(x^*)/T}$，使得：

$$\lim_{T \to 0} \pi_T(x^*) = 1$$

在溫度從無窮大下降的足夠慢時，可保證得到全域最佳解:

$$T_k = \frac{C}{\log(k + k_0)}, \quad \text{for sufficiently large } C > 0$$

b. Pseudo Code

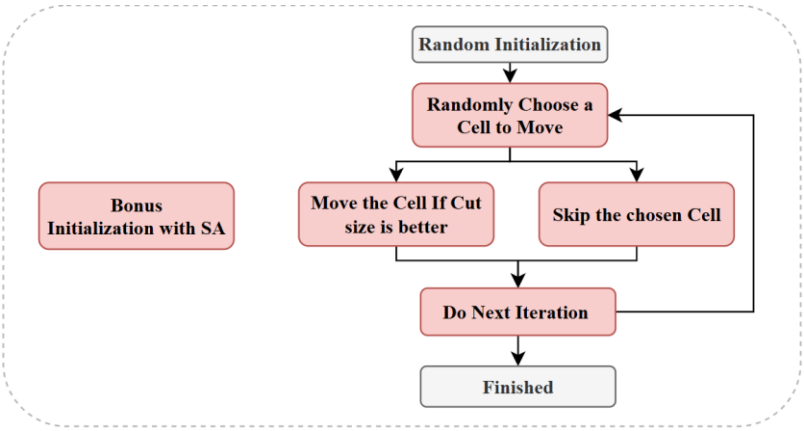
I. Formal definition of SA

- Let $s = s_0$
- For $k = 0$ through $k_{\max}$ (exclusive):
  - $T \leftarrow$ temperature(1 - $(k+1)/k_{\max}$)
  - Pick a random neighbour, $s_{\text{new}} \leftarrow$ neighbour(s)
  - If $P(E(s), E(s_{\text{new}}), T) \geq$ random(0, 1):
    - $s \leftarrow s_{\text{new}}$
- Output: the final state s

II. My Pseudo Code

```
Function initialize_with_SA(cell_set, net_set, partition, initialTemp, coolingRate, iterations):
    bestCut ← calculate_cutsize(net_set, partition)
    bestPartition ← partition
    (partA, partB) ← build A/B sets from partition
    temp ← initialTemp

    For i from 1 to iterations:
        Repeat 5 times:
            partition ← bestPartition
            (cellID, part) ← randomly pick one cell from partA or partB

            Flip cellID to opposite partition
            newCut ← calculate_cutsize(net_set, partition)
            delta ← newCut - bestCut

            If delta < 0:
                bestCut ← newCut
                bestPartition ← partition
                Update partA and partB sets accordingly
            Else:
                Revert the flip

            temp ← temp × coolingRate

        partition ← bestPartition

    Output final cutsize
End Function
```

c. Floe Chart of My SA Algorithm



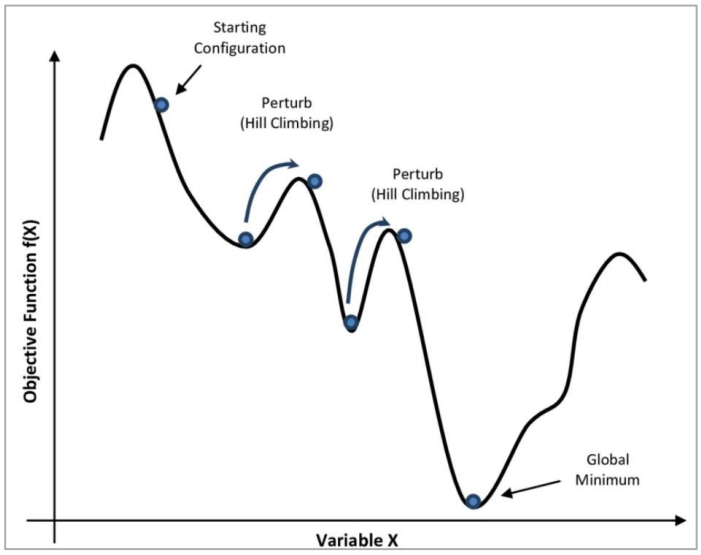
d. My code

```
int initialize_with_SA(unordered_map<int,struct cell_node>cell_set,
    unordered_map<int,struct net_node>net_set, unordered_map<int,string> &partition_result, double initialTemp, double coolingRate, int iterations){
    int cur_bestCutSize = calculate_cutsize(net_set,partition_result);
    unordered_set<int> part_A_cell_set;
    unordered_set<int> part_B_cell_set;
    construct_part_A_B_set(partition_result,part_A_cell_set, part_B_cell_set);
    double temp = initialTemp;
    srand(time(0));
    for(int i=0;i<3;i++){
        for (int iter = 0; iter < 1; iter++){      //選擇一個節點進行翻轉，以當前Partition為基礎去調整
            string chosen_part="None";
            int cell_index= get_random_cell_id_from_certain_part(cell_set, part_A_cell_set, part_B_cell_set,chosen_part);

            int gain_of_the_selected_cell=calculate_gain_of_cell_for_SA(cell_index,cell_set,net_set,partition_result);
            cout<<"gain_of_the_selected_cell: "<<gain_of_the_selected_cell<<endl;//已確認正確

            int newCutSize=cur_bestCutSize-gain_of_the_selected_cell;            //計算新的 cut-size
            int delta=-gain_of_the_selected_cell;   // 計算接受機率

            if(delta<0){                            // 接受新解(更新)
                cur_bestCutSize = newCutSize;
                if(chosen_part=="A"){
                    part_A_cell_set.erase(cell_index);
                    part_B_cell_set.insert(cell_index);
                    partition_result[cell_index]="B";
                }
                else{
                    part_B_cell_set.erase(cell_index);
                    part_A_cell_set.insert(cell_index);
                    partition_result[cell_index]="A";
                }
            }
            temp *= coolingRate;  // 降溫
        }
    }
    cout<<"cutsize after SA: "<<cur_bestCutSize<<endl;
    return cur_bestCutSize;
}
```

說明: 起始時先計算目前切割(cut-size)成本，並將當前分割結果視為最佳解。接著建立 A、B 兩分區的元件集合。演算法在每個迭代中隨機選取一個元件，嘗試將其從原本分區移至另一邊，並重新計算 cut-size。若新的 cut-size 更小，則接受此變動並更新最佳解與對應集合；否則恢復原本的狀態。每輪內部會執行 5 次元件變動嘗試，每次後將溫度乘上冷卻係數，模擬逐步降低探索程度的過程。完成所有外部迭代後，輸出最終 SA 優化後的 cut-size。此方法可在初始階段探索更多可能解，避免卡在局部最佳解，有助於後續 FM 演算法找到更好的整體解。

e. **Explanation**

利用 SA 得到 global optimal 是一個很棒的目標，且 SA 適用於解空間為離散的問題(例如分堆、排列、組合、以及圖論中多種經典問題)，但 SA 收斂至 global optimal 的充分條件是初始溫度趨近無窮且下降速度夠慢(換句話說就是 iteration 次數趨近無窮)，**實務上我們無法這樣處理 Partition 問題，也無法這樣處理其他解空間為離散的問題，所以利用 SA 做合理的初始化並將後續問題交給 Heuristic 的演算法(FM)是比較合適的作法，且 SA 接受較差的解的機率必須帶有隨機性，這也是實際上比較不容易掌握的。**



8. The Hardness of This Assignment

a. **PA2 的 Program Size 還是稍大，我也花了一些時間熟悉 Doubly Linked List，Bucket List 實作上的細節還是蠻多的，例如每次移動完 Cell 後 Max Gain Pointer 都要重找、用 Gain 推算 Cell node 在 Bucket List 中的 Index 也要計算正確(index=gain + max_gain)、每次移動 base cell 後原本所屬的 cell node map 和移動過去的 cell node map 都要更新，分別要 erase 以及 insert base cell。**

b. **FM 有一個很大的重點是要用夠快的速度計算 gain、cut size 與更新 gain，雖然沒有限制作法，但基本上要和教授提到的方式的時間複雜度差不多，不能差於上課所學的方式，否則很容易無法在時間內執行完 FM 並得到想要的結果。**

c. **這次 PA 讓我體會 EDA 對於演算法的高度要求，不確定是不是因為我沒上過演算法，老實說我很難在第一時間就想到時間複雜度的最優解，且因為每個 Net 的 cell 數不固定，我覺得這次作業每個 Function 的時間複雜度沒有很直觀，且步驟比較繁雜，都要稍微推一下。**

9. Reference

a. **Liu, L-T., et al. "A gradient method on the initial partition of Fiduccia-Mattheyses algorithm."** ***Proceedings of IEEE International Conference on Computer Aided Design (ICCAD)*****. IEEE, 1995.**

b. **Caldwell, A. E., Kahng, A. B., & Markov, I. L. (1999). Design and implementation of the fiduccia-mattheyses heuristic for vlsi netlist partitioning. In** ***Algorithm Engineering and Experimentation: International Workshop ALENEX'99 Baltimore, MD, USA, January 15–16, 1999 Selected Papers 1*** **(pp. 182-198). Springer Berlin Heidelberg.**

c. **Saab, Youssef. "Post-analysis-based clustering dramatically improves the Fiduccia-Mattheyses algorithm."** ***Proceedings of EURO-DAC 93 and EURO-VHDL 93-European Design Automation Conference*****. IEEE, 1993.**

d. **Li, Fujian.** ***A hardware/software co-design of the Fiduccia-Mattheyses partitioning algorithm*****. Diss. University of Guelph, 2006.**